

Teste Funcional

Validação do comportamento do sistema contra requisitos, regras de negócio e expectativas observáveis.

O que é Teste Funcional?

O Teste Funcional verifica se uma funcionalidade do sistema se comporta de acordo com requisitos, regras de negócio, critérios de aceite e necessidades do usuário. O foco está no comportamento observável: o QA fornece entradas, executa ações e compara o resultado obtido com o resultado esperado. Ele responde à pergunta: “o sistema está fazendo aquilo que deveria fazer?”. Um login, por exemplo, não deve ser validado apenas com credenciais corretas; também precisamos observar senha inválida, campos vazios, formatos incorretos, conta bloqueada, mensagens, redirecionamentos e persistência da sessão.

Caixa-preta: o comportamento importa

Testes funcionais são frequentemente associados a Black Box Testing. Não é obrigatório conhecer a implementação interna para validar o comportamento: Entrada → Sistema → Saída. Isso não significa que código, API, banco e arquitetura sejam irrelevantes; esse conhecimento ajuda a investigar defeitos. No teste funcional, porém, o objetivo principal continua sendo verificar o comportamento que o produto entrega.

O que queremos validar?

Valide fluxos, regras de negócio, campos obrigatórios, cálculos, mensagens, permissões, navegação, persistência, integrações percebidas pelo usuário e diferentes combinações de dados. Cubra caminho feliz, cenários alternativos, negativos e limites, priorizando o que representa maior risco.

Como pensar como QA

Transforme requisitos em perguntas. Em um login: e se a senha estiver errada? E-mail inexistente? Campos vazios? Espaços antes do e-mail? Conta bloqueada? Duplo clique em Entrar? Sessão expirada? Uma habilidade central de QA é sair de “vou testar o botão” para “vou testar o comportamento da funcionalidade”.

Exemplo prático — Carrinho de compras

Requisito: “o usuário pode adicionar produtos ao carrinho e alterar a quantidade antes de finalizar a compra”. Comece adicionando um produto e validando nome, quantidade, preço e subtotal. Depois explore: adicionar o mesmo item novamente, alterar 1 para 3, quantidade zero ou negativa, quantidade acima do estoque, remover produto, atualizar a página e retornar posteriormente.

Uma técnica simples para começar

Faça cinco perguntas: 1) O que deveria funcionar? 2) O que não deveria ser permitido? 3) Quais são os limites? 4) O que acontece se eu interromper o fluxo? 5) O que acontece depois? Para um campo com máximo de 100 caracteres, por exemplo, experimente 99, 100 e 101.

Transformando cenário em Caso de Teste

CT001 — Realizar login com credenciais válidas

Pré-condição: usuário cadastrado e ativo.

Dados de teste: e-mail cadastrado e senha válida.

Passos: 1) Acessar a página de login. 2) Informar o e-mail. 3) Informar a senha. 4) Clicar em Entrar. 5) Verificar a página apresentada.

Resultado esperado: o usuário é autenticado e direcionado corretamente para a área autenticada.

Casos derivados: CT002 — senha incorreta; CT003 — usuário inexistente; CT004 — campos vazios; CT005 — e-mail inválido; CT006 — conta bloqueada.

Quando encontramos um comportamento diferente

Se o esperado é permanecer no login e exibir “credenciais inválidas”, mas o sistema abre uma página em branco, temos Expected Result $\neq$ Actual Result. Investigue e, confirmado o defeito, registre um BUG com título, ambiente, pré-condição, passos, dados, resultado atual, resultado esperado e evidências.

Ferramentas que podem ajudar

Jira, Xray e TestRail ajudam na gestão; Postman apoia validações de API; DevTools ajuda com rede e console; SQL pode validar persistência; Playwright, Selenium ou Cypress podem automatizar cenários estáveis. Ferramentas ajudam a executar testes, mas não substituem o raciocínio do QA.

Checklist rápido

Antes de encerrar: fluxo principal; cenários alternativos; dados inválidos; obrigatoriedade; limites; mensagens de erro e sucesso; regras de negócio; estado após a ação; interrupções; permissões; persistência; situações não explicitadas no requisito.

QA Mindset

Não teste apenas aquilo que o sistema deveria fazer. Teste também aquilo que o usuário pode tentar fazer. Um bom teste funcional não é uma sequência grande de passos: é uma investigação orientada por boas perguntas sobre o comportamento do produto.